Process Mining: Recommendation System

Predictive Process Monitoring with Decision Trees

Giorgia Gabardi (257838)

Abstract

This project implements a recommendation system for predictive process monitoring based on decision tree classifiers. The system analyzes a real-world production event log to predict whether an ongoing process instance will lead to a positive outcome (fast: cycle time below average) or a negative outcome (slow: cycle time above average). For traces predicted as negative, recommendations are extracted from the decision tree, suggesting which activities should be performed, or avoided, to improve the expected outcome. The approach uses boolean encoding of prefix traces and includes a padding mechanism for shorter traces. Performance is evaluated both in terms of predictive accuracy (using standard classification metrics) and recommendation quality (using a dedicated evaluation metric that checks whether the recommended activities are actually followed in the complete traces). Results are compared across different prefix lengths to assess how early information availability affects both prediction and recommendation effectiveness.

Keywords: process mining, predictive monitoring, decision trees, recommendations, machine learning.

1 Introduction

In many real-world processes, it is useful to predict as early as possible whether an ongoing case will end positively or negatively, and to suggest corrective actions when a negative outcome is expected.

This work focuses on applying decision tree models to a real-world production log from a manufacturing setting with the goal of supporting predictive process monitoring. In particular, the study aims to:

- **Outcome Prediction**: Train a decision tree classifier to predict whether an ongoing process instance (represented as a trace prefix) will result in a positive or negative outcome.
- **Recommendation Generation**: For traces predicted to have negative outcomes, extract recommendations indicating which activities should be performed to achieve a positive result.
- **Evaluation**: Assess both the predictive accuracy of the classifier and the effectiveness of the generated recommendations.

The production log is labeled based on cycle time: traces with cycle time below the average are labeled as positive (*true*), while those above average are labeled as negative (*false*). The challenge is to predict this outcome using only partial trace information (prefixes) and to recommend corrective actions when necessary. Special attention is given to the handling of traces shorter than the chosen prefix length through proper padding, and to the exclusion of technical artifacts (such as trace identifiers and prefix length) from the feature space to ensure meaningful predictions. Finally, the prediction and recommendation results are compared across different prefix lengths to evaluate the trade-off between early intervention and information availability.

2 Implementation

The recommendation system was implemented in Python, using the *PM4Py* library for process mining operations and *scikit-learn* for machine learning. The code is organized into four modules:

- `preprocessing.py`: log import, prefix generation with padding, activity extraction, and boolean encoding.
- `types.py`: data structures for tree conditions (`BooleanCondition`, `ThresholdCondition`).
- `tree_utils.py`: DFS traversal, path confidence computation, compliance filtering, and missing condition extraction.
- `recommendations.py`: the main `extract_recommendations` and `evaluate_recommendations` functions.

The pipeline is executed in a Jupyter Notebook (`project.ipynb`) that sequentially calls each stage, from data loading to the final comparison across prefix lengths.

2.1 Data Loading and Preprocessing

2.1.1 Event Log Import

The event logs, provided in XES format, are imported using PM4Py and converted into `EventLog` objects for further processing. The dataset is pre-split into training (80%) and testing (20%) sets:

- `Production_avg_dur_training_0-80.xes`: Training set containing 177 traces
- `Production_avg_dur_testing_80-100.xes`: Test set containing 43 traces

2.1.2 Prefix Extraction with Padding

Since the goal is to make predictions on ongoing (incomplete) traces, each complete trace is truncated to a fixed *prefix_length* representing the number of events observed so far. This is important because it simulates the real-world scenario where decisions must be made before the process completes.

Padding Strategy: When working with trace prefixes, some process instances may contain fewer events than the selected prefix length. A padding mechanism is applied so that each trace is encoded with the same number of features. This choice prevents:

1. The exclusion of shorter traces from the analysis
2. A potential bias toward longer process instances
3. Inconsistencies in the structure of the feature matrix

The padding is implemented by adding special `PADDING` events to traces shorter than the required prefix length. The padding procedure is summarized in the following pseudocode:

Algorithm 1 Prefix Creation with Padding

Require: log ▷ Original event log
Require: $prefix\_length$ ▷ Target prefix length
Require: $use\_padding$ ▷ Boolean flag for padding shorter traces
Ensure: $prefixes\_log$ ▷ Event log with prefixes of uniform length

$prefixes\_log \leftarrow$ empty log
for each $trace$ in log **do**
  $prefix\_trace \leftarrow$ first $prefix\_length$ events of $trace$
  if $use\_padding$ AND length($prefix\_trace$) $<$ $prefix\_length$ **then**
    while length($prefix\_trace$) $<$ $prefix\_length$ **do**
      Append a special event `PADDING` to $prefix\_trace$
    end while
  end if
  Append $prefix\_trace$ to $prefixes\_log$
end for
return $prefixes\_log$

2.2 Activity Extraction and Boolean Encoding

2.2.1 Feature Space Definition

To construct the feature space, all unique activity names are extracted from the complete training log (rather than the prefixed version). This ensures that every possible activity in the process is represented as a feature, even if some activities do not appear in every trace. The extraction is performed by scanning each trace in the log and collecting activity names in the order they appear, while avoiding duplicates.

2.2.2 Boolean Encoding

Once the prefixed traces are created, they are transformed into a tabular format suitable for machine learning. In this representation, each row corresponds to a trace, and each column represents a specific activity from the feature space. A boolean value indicates whether the activity is present (True) or absent (False) in the trace prefix. Additional columns store the trace identifier, the prefix length and the ground truth label indicating the outcome of the trace.

2.2.3 Validation

After the boolean encoding, several checks are performed to ensure data integrity:

- All traces have a prefix length equal to the specified value.
- The total number of encoded traces matches the number of input traces.
- The special `PADDING` activity is correctly included for traces that required padding.

This encoding allows the decision tree model to process the traces as structured feature vectors, preserving information about the activities present in each prefix.

2.3 Feature Engineering and Training Preparation

2.3.1 Feature Selection

An important choice is which columns to use as features for the decision tree. The encoded DataFrame contains:

- `trace_id`: Technical identifier (must be excluded)
- `prefix_length`: Length of the prefix (must be excluded)
- Activity columns: Boolean indicators (features for training)
- `label`: Ground truth (target variable)

Note: The `prefix_length` column is included in the encoded DataFrame for validation purposes, but it should not be used as a feature. It represents a technical artifact rather than a characteristic of the process, is constant across traces when padding is applied, has no meaningful connection to the outcome and could prevent the model from generalizing to different prefix lengths.

2.4 Decision Tree Training and Hyperparameter Optimization

The feature matrix X is constructed by selecting only the boolean activity columns from the encoded DataFrame, while the columns `trace_id`, `prefix_length` and `label` are excluded. The label column is used as the target variable y. The same procedure is applied to both the training and test sets, and consistency checks are performed to ensure dimensional alignment and that no technical artifact is included among the features.

2.4.1 Hyperparameter Optimization

Before training the final model, a grid search with 5-fold cross-validation (`GridSearchCV`) is optionally performed to identify the best combination of hyperparameters. The search space includes:

- `max_depth`: controls the maximum depth of the tree, limiting its complexity.
- `criterion`: the splitting criterion (*gini* impurity or *entropy*).
- `max_features`: the number of features considered at each split (*None*, *sqrt*, *log2*).
- `min_samples_split` and `min_samples_leaf`: minimum number of samples required to split an internal node and to form a leaf, respectively.

All combinations are evaluated exhaustively, and the configuration achieving the highest F1-score across the cross-validation folds is selected. A fixed random seed is used throughout to ensure reproducibility.

2.4.2 Model Training

If hyperparameter optimization is applied, the best parameters found by the grid search are used; otherwise, a set of default parameters is used (e.g., limited tree depth, Gini criterion). The decision tree is then trained on the full training set using scikit-learn's `DecisionTreeClassifier`. After training, the resulting tree depth and number of leaves are reported to assess the model's complexity.

2.5 Decision Tree Visualization

After training, the decision tree is exported and visualized using the `graphviz` library. This visualization helps understand the internal logic of the model: it shows the feature used at each decision node, the split threshold, the class distribution at each node and the predicted class at each leaf.

Since the features are boolean (activity presence), each internal node represents a question of the form *"Is activity X present in the trace prefix?"* The left branch corresponds to *False* (activity absent) and the right branch corresponds to *True* (activity present). The leaves display the predicted class (*true* for fast traces, *false* for slow traces) along with the proportion of training samples reaching that node. The tree is rendered with colored nodes: blue shades indicate majority negative class, orange shades indicate majority positive class, with intensity proportional to class purity.

2.6 Prediction and Model Evaluation

2.6.1 Test Set Predictions

The trained decision tree classifier is used to generate predictions on the test set. Each test trace prefix, encoded using the same boolean encoding scheme as the training set, is classified as either *true* (fast) or *false* (slow):

$$\hat{y}_i = \text{clf.predict}(\mathbf{x}_i), \quad \forall\, i \in \{1, \ldots, n_{\text{test}}\}$$

2.6.2 Performance Metrics

Standard classification metrics are computed to evaluate the predictive accuracy of the model. For a binary classification task with positive label *true* and negative label *false*, the following metrics are used:

- **Accuracy**: fraction of correctly classified instances.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision**: fraction of predicted positives that are truly positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall**: fraction of actual positives that are correctly identified.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score**: harmonic mean of precision and recall.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

2.6.3 Confusion Matrix

The confusion matrix is computed with explicit label ordering (`labels=['false', 'true']`) to ensure correct interpretation of the results:

		Predicted	
		Negative	Positive
Actual	Negative (*false*)	TN	FP
	Positive (*true*)	FN	TP

Where:

- **TN (True Negatives)**: Slow traces correctly predicted as slow.
- **FP (False Positives)**: Slow traces incorrectly predicted as fast.
- **FN (False Negatives)**: Fast traces incorrectly predicted as slow.
- **TP (True Positives)**: Fast traces correctly predicted as fast.

2.7 Recommendation Generation

The main part of this project is the generation of recommendations for traces predicted as negative. The goal is to suggest which activities should be performed (or avoided) to push the process towards a positive outcome. The recommendation algorithm uses the internal structure of the trained decision tree.

2.7.1 Data Structures

Two types of conditions are used to represent the decision rules extracted from the tree:

- **BooleanCondition**: Represents a condition on a boolean feature (activity), defined by the activity name and the required value (*True* = activity should be present, *False* = activity should be absent).
- **ThresholdCondition**: Represents a numerical condition (used internally for `prefix_length` if it were included as feature), defined by the feature name, a comparison operator ($\leq$ or $>$) and a threshold value.

A *path* is defined as a sequence of conditions from the root of the tree to a leaf node. Each path encodes a set of requirements that, if satisfied, lead to the class predicted by the corresponding leaf.

2.7.2 Algorithm Overview

The recommendation extraction process follows four main steps:

1. **Extract Positive Paths**: Traverse the decision tree using Depth-First Search (DFS) to identify all root-to-leaf paths that lead to a positive prediction.
2. **Filter Compliant Paths**: For each negatively predicted trace, select only the positive paths whose conditions do not contradict the current state of the trace.
3. **Select Best Path**: Among the compliant paths, choose the one with the highest confidence (and shortest length in case of ties).
4. **Extract Missing Conditions**: Compare the selected path's conditions with the trace's current state to identify the activities that need to change.

2.7.3 Step 1: Extracting Positive Paths via DFS

The decision tree is traversed using a recursive Depth-First Search starting from the root node. At each internal node, the algorithm records the splitting condition (which activity is tested and in which direction) and recurses into both children. When a leaf node is reached, the algorithm checks whether the predicted class is positive. If so, the accumulated path of conditions is saved together with a confidence score.

The **confidence** of a leaf node ℓ is defined as the class probability of the positive class at that leaf:

$$\text{confidence}(\ell) = \frac{n_{\text{positive}}(\ell)}{n_{\text{samples}}(\ell)}$$

where $n_{\text{positive}}(\ell)$ is the number of positive-class samples in the leaf and $n_{\text{samples}}(\ell)$ is the total number of training samples reaching that leaf. In other words, it is the fraction of training instances at the leaf that belong to the positive class.

Note that since `prefix_length` is excluded from the features, the DFS encounters only boolean splits. However, the implementation also supports threshold-based splits (via `ThresholdCondition`) for generality.

Algorithm 2 Extract Positive Paths (DFS)

Require: *tree* ▷ Trained Decision Tree
Require: *feature_names* ▷ List of feature names
Require: *positive_class_idx* ▷ Index of positive class in tree.classes_
Ensure: *paths* ▷ List of (path, confidence) for positive leaves

paths ← empty list
function DFS(*node*, *current_path*)
  if *node* is a leaf **then**
    if arg max of class counts at *node* = *positive_class_idx* **then**
      conf ← n_{positive}(*node*) / n_{samples}(*node*)
      Append (*current_path*, *conf*) to *paths*
    end if
    return
  end if
  feat ← feature at *node*
  DFS(left child, *current_path* + [BooleanCondition(*feat*, False)])
  DFS(right child, *current_path* + [BooleanCondition(*feat*, True)])
end function
DFS(root, empty list)
return *paths*

2.7.4 Step 2: Filtering Compliant Paths

Given a trace predicted as negative, not all positive paths are reachable: some may require conditions that contradict the current state of the trace. A path is said to be *compliant* with a trace if none of its conditions contradict the trace's current feature values.

A condition *contradicts* a trace if the condition requires an activity to have a specific value (present or absent), but the trace already has the opposite value. Crucially, if a condition requires an activity that is *not yet present* in the trace, this is not considered a contradiction—it represents a potential future action that could still happen.

Formally, let $P = \{c_1, c_2, \ldots, c_k\}$ be a positive path and $\mathbf{t}$ the current trace state. The path is compliant if:

$$\forall\, c_i \in P :\ \neg\,\text{contradicts}(c_i, \mathbf{t})$$

2.7.5 Step 3: Selecting the Best Path

Among all compliant paths, the algorithm selects the one that offers the highest confidence. In case of ties, the path with fewer conditions (i.e., shorter length) is preferred, as it requires fewer changes to the trace:

$$P^* = \arg\max_{P \in \mathcal{C}} \left(\text{confidence}(P),\ -|P|\right)$$

where $\mathcal{C}$ is the set of compliant paths and $|P|$ is the number of conditions in the path.

2.7.6 Step 4: Extracting Missing Conditions (Recommendations)

Once the best compliant path is selected, the algorithm compares each of its boolean conditions against the trace's current state. For each condition, the actual value of the activity in the trace is determined: if the activity is present in the trace's feature dictionary it is *True*; if it is absent (not in the dictionary), it defaults to *False*. A condition is *missing* if the required value differs from the actual value:

$$\text{Recommendations}(\mathbf{t}, P^*) = \{c \in P^* \mid c.\text{value} \neq \mathbf{t}.\text{get}(c.\text{feature},\ \text{False})\}$$

Each recommendation is a `BooleanCondition` indicating:

- If `value = True`: the activity should be **added** (i.e., performed in the future steps of the process).
- If `value = False`: the activity should be **avoided** (i.e., it is detrimental to a positive outcome).

2.7.7 Complete Recommendation Pipeline

The full recommendation pipeline is implemented in the function `extract_recommendations()`, summarized in Algorithm 3.

This function takes as input the decision tree, the feature names, the class values, and the prefix set.

Algorithm 3 Extract Recommendations

Require: *tree* ▷ Trained Decision Tree
Require: *feature_names* ▷ Feature names
Require: *class_values* ▷ [*positive_class*, *negative_class*]
Require: *prefix_set* ▷ Boolean-encoded test set
Ensure: *recommendations* ▷ Dictionary mapping traces to recommendations

positive_class ← *class_values*[0]
positive_class_idx ← index of *positive_class* in *tree.classes_*
positive_paths ← ExtractPositivePaths(*tree*, *feature_names*, *positive_class_idx*)
predictions ← *tree*.predict(*prefix_set*[*feature_names*])
recommendations ← empty dictionary
for each prefix trace $\mathbf{t}_i$ in *prefix_set* **do**
    $key_i \leftarrow i$ ▷ positional index of trace $\mathbf{t}_i$ in *prefix_set*
    if *predictions*[*i*] = *positive_class* **then**
        recommendations[key_i] ← ∅ ▷ Already positive, empty set
        continue
    end if
    current_state ← activities with value True in $\mathbf{t}_i$
    compliant ← FilterCompliantPaths(*positive_paths*, *current_state*)
    if *compliant* is empty **then**
        recommendations[key_i] ← ∅ ▷ No viable path, empty set
        continue
    end if
    $P^* \leftarrow \arg\max_{P \in compliant}(\text{confidence}(P), -|P|)$
    missing ← GetMissingConditions(P^*, *current_state*)
    recommendations[key_i] ← *missing*
end for
return *recommendations*

The function predicts the label internally using the tree, so the input `prefix_set` only needs to be the boolean-encoded DataFrame (including `trace_id`, `label`, `prefix_length` columns, which are ignored during prediction). The output is a dictionary where each key is the positional index i of the trace in the prefix set and the associated value is:

- An **empty set**: if the trace was predicted as positive (no recommendation needed), or if no compliant positive path was found (no viable recommendation).
- A **set of BooleanCondition** objects: the recommended changes to reach a positive outcome.

2.8 Recommendation Evaluation

To assess the quality of the generated recommendations, the function `evaluate_recommendations` compares the recommended activities against the *complete* (non-truncated) test traces. The intuition is: if the system recommends that an activity should be present, we check whether that activity *actually occurs* in the full trace and whether the trace's ground-truth outcome is positive.

2.8.1 Evaluation Criteria

The evaluation is performed only on traces with a **negative prediction** (since positive predictions receive no recommendation). For each such trace, the algorithm:

1. Looks up the recommendation in the dictionary (using the trace's positional index as key).
2. Boolean-encodes the *complete* test trace (not just the prefix).
3. Checks whether **all recommended conditions are satisfied** in the complete trace:

- For each condition with `value = True`: the activity must be present in the full trace.
- For each condition with `value = False`: the activity must be absent from the full trace.

4. If all conditions are met, the recommendation is considered *followed*; otherwise, it is *not followed*.
5. If the recommendation set is empty (no recommendation could be generated), it is classified as *not followed*.

Based on this, each evaluated trace is classified into one of four categories:

		Ground Truth Outcome	
		Negative (*false*)	Positive (*true*)
Recommendation	Followed	FP	TP
	Not Followed	TN	FN

In this context, the confusion matrix entries have a different interpretation than for standard classification: **TP** indicates a recommendation that was followed and led to a positive outcome; **TN** indicates a recommendation not followed with a negative outcome; **FP** indicates a followed recommendation that did not prevent a negative outcome; **FN** indicates a trace that achieved a positive outcome without following the recommendation.

2.8.2 Recommendation Metrics

From this classification, the same standard metrics defined in Section 2.6.2 (Accuracy, Precision, Recall, F1-Score) are computed to obtain what we call the *recommendation F-measure* (F_{rec}). These metrics measure how well the recommendation system performs: high F_{rec} indicates that the recommended activities were actually followed and led to positive outcomes.

2.9 Qualitative Inspection of Recommendations

Beyond quantitative evaluation, the notebook provides a qualitative analysis of the generated recommendations.

2.9.1 Textual Analysis

For each negatively predicted trace that received a recommendation, the system prints:

- The set of activities currently present in the prefix.
- The recommended actions: activities to **add** (perform in the future) or to **remove** (avoid).

A detailed example is presented for a specific trace, showing the mapping between the trace's current state and the suggested modifications.

2.9.2 Visual Example on the Decision Tree

To illustrate how recommendations relate to the tree's structure, a selected negative trace is visualized directly on the decision tree. The visualization highlights:

- The **path of the trace** through the tree (orange border), showing which nodes are traversed given the trace's current feature values.
- The **recommendation nodes** (blue border for ADD, red border for REMOVE), indicating the tree nodes where the recommended features are tested.
- The **leaf node** where the trace ends up (negative prediction).

This visualization shows clearly *why* the trace received a negative prediction and *what* the tree suggests to fix it. It also serves as a check to verify that the recommendation algorithm correctly identifies the relevant decision boundaries.

2.10 Prefix Length Comparison

To evaluate how the amount of available information affects both prediction and recommendation quality, the entire pipeline described above—from prefix generation through recommendation evaluation—is executed independently for every prefix length from 1 to 12. For each prefix length, a new decision tree is optimized via GridSearchCV and trained from scratch; predictions, recommendations, and evaluation metrics are collected into a summary dictionary.

3 How to Run

This section provides step-by-step instructions to reproduce the results presented in this report.

3.1 Prerequisites

The implementation requires Python 3.x and Jupyter Notebook. All dependencies are listed in the `requirements.txt` file located in the `implementation` folder.

3.2 Setup Instructions

1. **Install dependencies**: Navigate to the `implementation` folder and install the required Python packages:

```
cd implementation
pip install -r requirements.txt
```

2. **Prepare the event logs**: Ensure that the two XES files are placed in the root directory of the project:
   - `Production_avg_dur_training_0-80.xes` (training set)
   - `Production_avg_dur_testing_80-100.xes` (test set)

   These files should be located one level above the `implementation` folder.

3. **Run the notebook**: Open and execute `project.ipynb` located in the `implementation` folder:

```
jupyter notebook project.ipynb
```

4. **Execute all cells**: Run all cells sequentially from top to bottom. The notebook is divided into steps that cover:
   - loading libraries and event logs;
   - preprocessing and boolean encoding;
   - training the decision tree classifier;
   - making predictions and extracting recommendations;
   - evaluating recommendations and comparing prefix lengths.

3.3 Project Structure

The project is organized as follows:

```
PROCESS-MINING-PROJECT_Providing-recommendations/
|-- Production_avg_dur_training_0-80.xes
|-- Production_avg_dur_testing_80-100.xes
|-- implementation/
    |-- project.ipynb
    |-- requirements.txt
    |-- src/
```

```
        |-- preprocessing.py
        |-- tree_utils.py
        |-- recommendations.py
        |-- types.py
```

4 Results

This section presents the experimental results obtained by applying the recommendation pipeline to the production event log. All results are reported for the default prefix length of 11, unless otherwise stated.

4.1 Dataset Characteristics

The production event log contains 220 traces in total, pre-split into a training set of 177 traces and a test set of 43 traces. Each trace represents a production process instance, labeled as *true* (fast: cycle time below average) or *false* (slow: cycle time above average).

	Training	**Test**	**Total**
Positive (*true*)	116 (65.5%)	32 (74.4%)	148 (67.3%)
Negative (*false*)	61 (34.5%)	11 (25.6%)	72 (32.7%)
Total	177	43	220

The dataset is imbalanced in favor of the positive class (∼65–75%), which is an important consideration for both model training and evaluation. After prefix extraction with padding at length 11, the boolean encoding produces 26 activity features per trace (plus `trace_id`, `prefix_length`, and `label`, which are excluded from training). The 26 distinct activities include machine-specific operations (e.g., *Turning & Milling – Machine 8*), quality control steps (e.g., *Turning & Milling Q.C.*, *Final Inspection Q.C.*), and an *other* category.

4.2 Hyperparameter Optimization

GridSearchCV explored 216 parameter combinations over a 5-fold cross-validation with F1-score (with the positive label set to 'true') as the optimization criterion. The search space was:

Hyperparameter	**Values**
`criterion`	`gini`, `entropy`
`max_depth`	2, 3, 4, 5
`max_features`	`None`, `sqrt`, `log2`
`min_samples_split`	2, 5, 10
`min_samples_leaf`	1, 2, 4

The best parameters found for prefix length 11 are:

Parameter	**Best Value**
`criterion`	`entropy`
`max_depth`	2
`max_features`	`None`
`min_samples_leaf`	1
`min_samples_split`	2

The best cross-validated F1-score on the training set was **0.822**. The selection of `max_depth` = 2 indicates that the optimal model is a very shallow tree with only 4 leaves, which favors interpretability and avoids overfitting. The choice of `entropy` as the splitting criterion suggests that information gain provides better class separation for this dataset.

4.3 Decision Tree Prediction Performance

The trained decision tree (depth 2, 4 leaves) was evaluated on the 43 test traces with prefix length 11. The confusion matrix and derived metrics are reported below.

4.3.1 Confusion Matrix

		Predicted		
		Negative	Positive	**Total**
Actual	Negative (*false*)	5	6	11
	Positive (*true*)	2	30	32
	Total	7	36	43

4.3.2 Classification Metrics

Metric	**Value**
Accuracy	0.814
Precision	0.833
Recall	0.938
F1-Score	0.882

The model achieves an accuracy of 81.4% and an F1-score of 0.882 on the test set. The model achieves a high recall of 0.938 which means that the model correctly identifies 93.8% of the truly fast traces. The precision of 0.833 indicates that among the 36 traces predicted as positive, 30 are actually positive while 6 are false positives (slow traces incorrectly predicted as fast). Only 2 fast traces are missed (false negatives), confirming the model's strength in detecting positive outcomes. The relatively high number of false positives (6 out of 11 actual negatives) is likely due to the dataset imbalance and the limited depth of the tree. In practice, a false positive means a trace predicted as fast does not receive a recommendation. This is less problematic than a false negative, where a fast trace would receive an unnecessary recommendation.

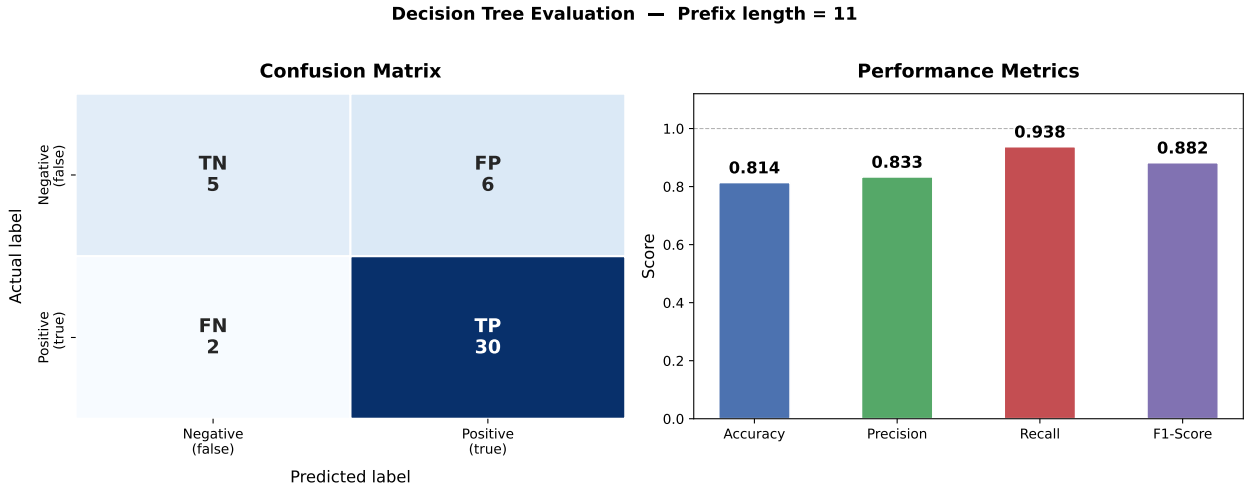

Figure 1: Decision tree evaluation on the test set (prefix length = 11). **Left**: Confusion matrix showing the distribution of predictions. **Right**: Bar chart of the four classification metrics. The model achieves high recall (0.938) at the cost of moderate precision (0.833).

4.4 Tree Structure and Key Splitting Features

The optimized tree has depth 2 with 4 leaf nodes. The root node splits on **Laser Marking – Machine 7**: traces that include this activity are routed to the right subtree, while those without it go to the left. At depth 2:

- **Left subtree** (Laser Marking – Machine 7 = False): splits on **Turning Q.C.**
  - No Turning Q.C. ⇒ Leaf: *true* (positive)
  - With Turning Q.C. ⇒ Leaf: *false* (negative)
- **Right subtree** (Laser Marking – Machine 7 = True): splits on **Final Inspection Q.C.**
  - No Final Inspection Q.C. ⇒ Leaf: *false* (negative)
  - With Final Inspection Q.C. ⇒ Leaf: *true* (positive)

This structure reveals that *Laser Marking – Machine 7* is the most discriminative feature at the process level. Among traces that pass through laser marking, the presence of *Final Inspection Q.C.* is the key differentiator for a positive outcome. Among traces that do not involve laser marking, the absence of *Turning Q.C.* is associated with faster completion.

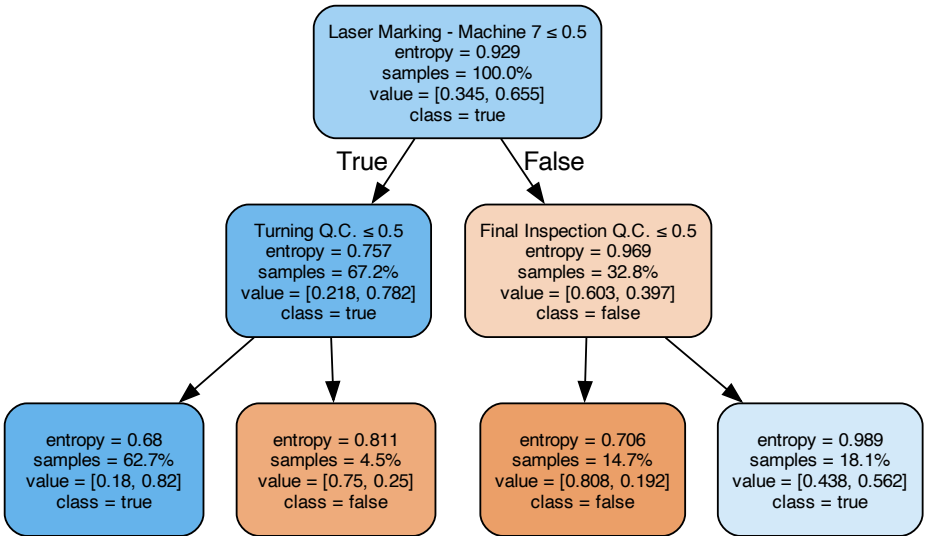

Figure 2: Decision tree trained with optimized parameters (`criterion=entropy`, `max_depth=2`).

4.5 Recommendation Results

4.5.1 Summary Statistics

Out of 43 test traces, the decision tree predicted 36 as positive and 7 as negative. The recommendation algorithm was applied to all 7 negatively predicted traces:

Category	**Count**
Positive predictions (no recommendation)	36
Negative predictions with recommendations	7
Negative predictions without viable path	0
Total	43

All 7 negatively predicted traces received at least one recommendation, meaning the algorithm always found a compliant positive path through the tree. The DFS traversal identified 2 positive paths in the tree (corresponding to the 2 positive leaves).

4.5.2 Content of the Recommendations

The recommendations generated for the 7 negative traces converge on a consistent pattern:

- **4 traces** received a single recommendation: **Add *Final Inspection Q.C.*** These are traces that already contain *Laser Marking – Machine 7* but lack the quality control step.
- **3 traces** received two recommendations: **Add *Laser Marking – Machine 7*** and **Add *Final Inspection Q.C.*** These are traces that contain *Turning Q.C.* but lack both key activities from the positive paths.

This is consistent with the tree structure: the two positive leaves require either (i) *Laser Marking – Machine 7* = True and *Final Inspection Q.C.* = True, or (ii) *Laser Marking – Machine 7* = False and *Turning Q.C.* = False. Since the algorithm cannot recommend *removing* an activity already present in the prefix (that would be a contradiction), traces with *Turning Q.C.* = True are directed towards the first positive path, which requires adding both *Laser Marking – Machine 7* and *Final Inspection Q.C.*

4.5.3 Detailed Example

Consider a trace whose prefix contains the following 7 activities:

{Lapping – Machine 1, Laser Marking – Machine 7, Round Grinding – Machine 3,
Turning & Milling – Machine 4, Turning & Milling – Machine 5,
Turning & Milling Q.C., other}

This trace is routed through the tree as follows:

1. **Node 0** (root): Split on *Laser Marking – Machine 7* → True → go right.
2. **Node 4**: Split on *Final Inspection Q.C.* → False → go left.
3. **Node 5**: Leaf → predicted class: *false* (negative).

The trace ends in a negative leaf because it lacks *Final Inspection Q.C.* The recommendation is straightforward: **Add *Final Inspection Q.C.*** If this quality control step is performed in the remaining process steps, the trace would follow the path to the positive leaf (Node 6) instead.

Figure 3 visualizes this example directly on the decision tree. The orange border highlights the path traversed by the trace through the tree and the blue one highlights the node where Final Inspection Q.C. is evaluated, which is the recommended feature. This visualization confirms that the recommendation targets exactly the decision boundary that separates the negative leaf (Node 5) from the positive leaf (Node 6).

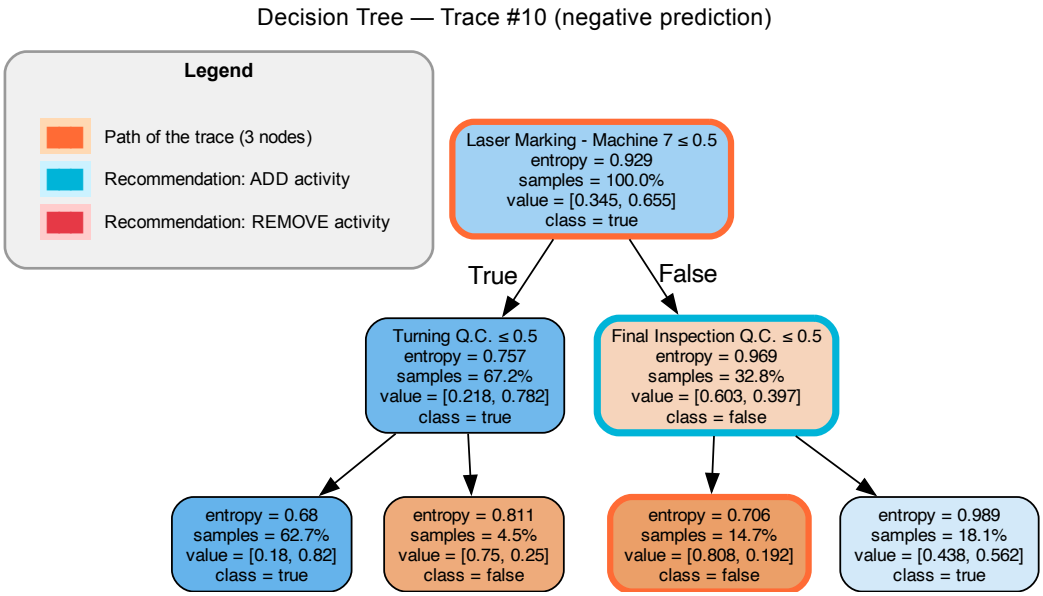

Figure 3: Annotated decision tree for a selected negative trace.

4.6 Recommendation Evaluation

The recommendations were evaluated against the complete (non-truncated) test traces and this produced the following confusion matrix:

		Ground Truth	
		Negative	Positive
Recommendation	Followed	1 (FP)	1 (TP)
	Not Followed	4 (TN)	1 (FN)

Metric	**Value**
$\text{Accuracy}_{\text{rec}}$	0.714
$\text{Precision}_{\text{rec}}$	0.500
$\text{Recall}_{\text{rec}}$	0.500
F_{rec}	0.500

The recommendation accuracy is 71.4%, meaning that 5 out of 7 traces were classified correctly. Precision and recall are both 0.500, which is likely due to the small number of traces used in the evaluation. Specifically:

- **TP = 1**: One trace followed the recommendation (the recommended activity was present in the complete trace) and indeed had a positive ground-truth outcome.
- **TN = 4**: Four traces did not follow the recommendation (the recommended activity was absent in the complete trace) and their ground-truth outcome was negative. These represent cases where the slow traces remained slow, consistent with not following the advice.
- **FP = 1**: One trace followed the recommendation but the ground-truth outcome was still negative. This indicates the recommendation was not sufficient to change the outcome.

- **FN = 1**: One trace did not follow the recommendation but still achieved a positive outcome, suggesting other (unmodeled) factors contributed to the positive result.

With larger datasets the metrics would likely stabilize and provide more reliable estimates.

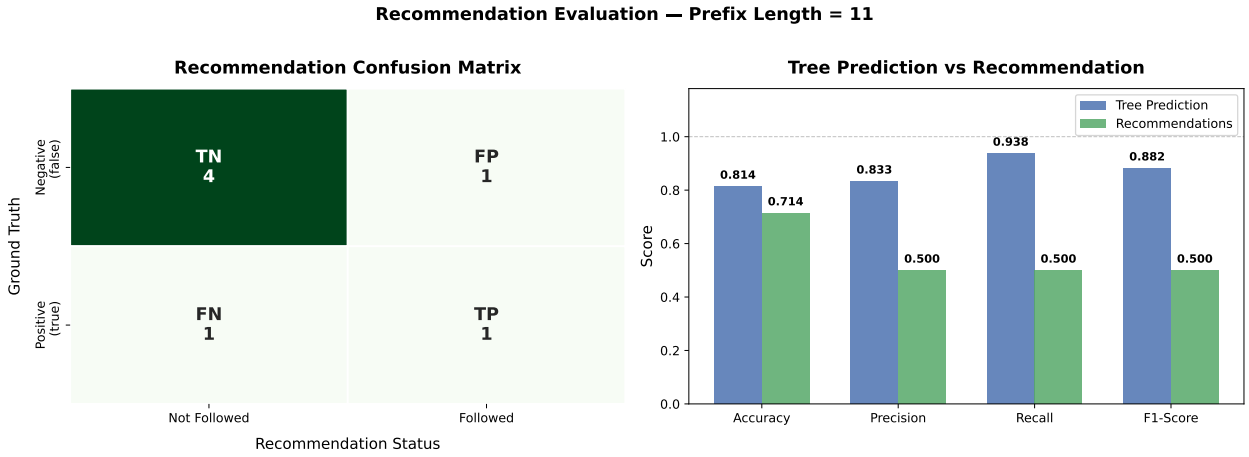

Figure 4: Recommendation evaluation results (prefix length = 11). **Left**: Confusion matrix for the recommendation system, where the axes represent the recommendation status (followed/not followed) and the ground-truth outcome. **Right**: Side-by-side comparison of tree prediction metrics (blue) and recommendation evaluation metrics (green). The gap between the two sets of metrics reflects the additional difficulty of not only predicting the outcome correctly, but also generating recommendations that are actually followed and effective.

4.7 Prefix Length Comparison

To assess the impact of prefix length on both prediction and recommendation quality, the entire pipeline was executed independently for every prefix length from 1 to 12. Table 3 reports the F1-score for both the decision tree classifier and the recommendation system, together with the number of negatively predicted traces (which determines how many traces are eligible for recommendation evaluation).

Table 1 provides a direct side-by-side comparison of the two representative prefix lengths suggested by the assignment (5 and 11), reporting all four metrics for both the decision tree classifier and the recommendation system.

Table 1: Direct comparison of prefix lengths 5 and 11. All metrics are reported for both the decision tree prediction and the recommendation evaluation.

	Decision Tree		**Recommendations**	
Metric	**Prefix 5**	**Prefix 11**	**Prefix 5**	**Prefix 11**
Accuracy	0.744	0.814	0.000	0.714
Precision	0.744	0.833	0.000	0.500
Recall	1.000	0.938	0.000	0.500
F1-Score	0.853	0.882	0.000	0.500
Neg. Predictions	0	7	—	—

The confusion matrices for the two prefix lengths are shown in Table 2.

Table 2: Confusion matrices for the decision tree classifier at prefix lengths 5 and 11.

Prefix length = 5

		Predicted	
		Neg.	Pos.
Actual	Neg.	0	11
	Pos.	0	32

Prefix length = 11

		Predicted	
		Neg.	Pos.
Actual	Neg.	5	6
	Pos.	2	30

At prefix length 5, the tree predicts *all* 43 traces as positive (zero negative predictions), correctly identifying all 32 fast traces (recall = 1.0) but misclassifying all 11 slow traces as fast. With only 5 events observed, the activities in the prefix are not yet informative enough for the tree to distinguish slow from fast traces, so it defaults to predicting the majority class (positive). As a consequence, no recommendations can be extracted or evaluated, resulting in zero for all recommendation metrics.

At prefix length 11, the tree achieves a more balanced confusion matrix: it correctly identifies 5 out of 11 negative traces (TN = 5) while maintaining high recall on positives (30/32). The 7 negative predictions enable the recommendation system to operate, achieving $F_{\text{rec}} = 0.500$. This comparison shows that short prefixes do not provide enough information to identify at-risk cases, while longer prefixes allow both better classification and useful recommendations.

The full results across all prefix lengths 1–12 are reported in Table 3.

Table 3: Decision tree and recommendation F1-scores across prefix lengths 1–12. "Neg. Preds" indicates the number of test traces predicted as negative, i.e., the traces for which recommendations are generated and evaluated.

Prefix	**Tree Acc.**	**Tree F1**	**Rec. F1**	**Neg. Preds**
1	0.744	0.853	0.000	0
2	0.744	0.849	0.000	2
3	0.744	0.853	0.000	0
4	0.744	0.853	0.000	0
5	0.744	0.853	0.000	0
6	0.721	0.838	0.000	1
7	0.744	0.853	0.000	0
8	0.744	0.853	0.000	0
9	0.744	0.825	0.250	12
10	0.744	0.841	0.000	6
11	0.814	0.882	0.500	7
12	0.767	0.839	0.000	13

4.7.1 Decision Tree F1 Trend

The left panel of Figure 5 shows that the decision tree F1-score is fairly stable across prefix lengths, around 0.85. For short prefixes (1–8), the tree tends to predict the majority class (positive), achieving high F1 primarily through high recall but low discriminative power (0 or very few negative predictions). The peak is reached at **prefix length 11** (F1 = 0.882), where the tree achieves the best balance between precision (0.833) and recall (0.938). At prefix length 12, the F1 decreases slightly to 0.839, suggesting that additional events may introduce noise or reduce generalization.

4.7.2 Recommendation F1 Trend and Negative Predictions

The center panel of Figure 5 shows a clear pattern: the recommendation F1 is **zero for most prefix lengths** (1–8), with non-trivial values appearing only from prefix length 9 onward. The right panel explains why: it shows the number of negative predictions at each prefix length. For short prefixes (1–8), the tree predicts

almost all traces as positive, producing 0 or very few negative predictions and therefore leaving no traces for which to generate and evaluate recommendations.

The recommendation system achieves its best performance at **prefix length 11** ($F_{\text{rec}} = 0.500$), coinciding with the prefix length where the decision tree also achieves its highest F1. At prefix length 9, a moderate F_{rec} of 0.250 emerges alongside the first substantial wave of negative predictions (12 traces).

Prefix lengths 10 and 12, however, show a different pattern: despite generating 6 and 13 negative predictions respectively, their recommendation F-score is 0.000. A recommendation is evaluated as a true positive only if the recommended activities actually appear in the complete trace and the ground truth is positive. A score of 0.000 therefore indicates that, for those prefix lengths, none of the negatively-predicted traces satisfied both conditions simultaneously—either the recommended activities did not appear in the remaining events of the trace, or the trace remained slow. This suggests that the quality of recommendations depends not only on the number of negative predictions, but also on whether the specific tree structure and observable features at a given prefix length produce recommendations aligned with the actual process behaviour.

4.7.3 Key Insights

The prefix length analysis shows the following:

- **Prediction stability**: The decision tree F1-score is relatively insensitive to prefix length, remaining in the 0.82–0.88 range.
- **Recommendation threshold**: Meaningful recommendations only emerge from prefix length 9 onward, when the tree begins to make a non-trivial number of negative predictions. Without enough negative predictions, there are simply no traces to recommend on.
- **Optimal prefix length**: Prefix length 11 is the sweet spot for this dataset, maximizing both the tree F1-score (0.882) and the recommendation F1-score (0.500).
- **Non-monotonic behavior**: The recommendation F1 does not increase monotonically with prefix length. Having more negative predictions (prefix 12: 13 negatives) does not guarantee better recommendation quality, suggesting that the specific tree structure and the activities observable at each prefix length play a critical role.
- **Negative prediction concentration**: As visible in the right panel of Figure 5, negative predictions are heavily concentrated at prefix lengths 9–12, with a sharp jump from 0–2 (prefixes 1–8) to 6–13 (prefixes 9–12). This transition marks the point where the process has revealed enough activities for the tree to start distinguishing slow traces from fast ones.

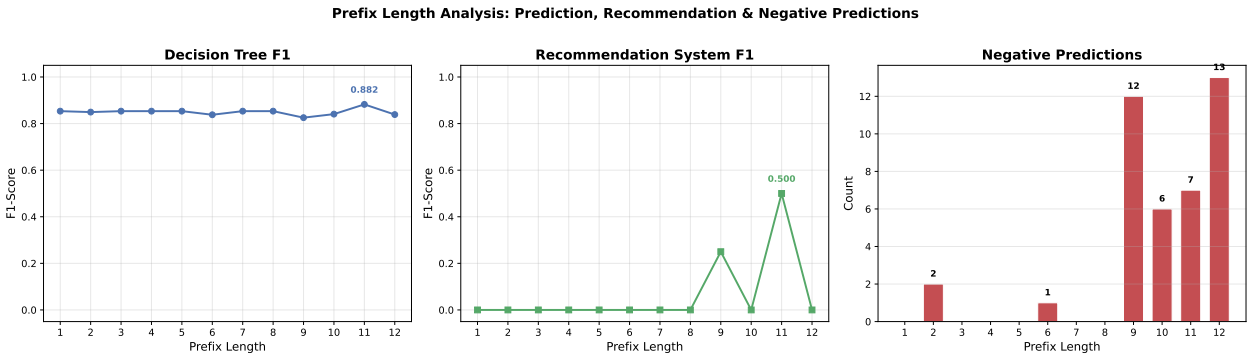

Figure 5: Prefix length analysis (1–12). **Left**: Decision tree F1—relatively stable around 0.85, peaking at prefix length 11 (0.882). **Center**: Recommendation system F1—zero for most short prefixes, with the best performance at prefix length 11 ($F_{\text{rec}} = 0.500$). **Right**: Number of negative predictions per prefix length—the bar chart explains the recommendation trend: only when the tree produces a meaningful number of negative predictions (prefix ≥ 9) can the recommendation system be evaluated and potentially succeed.

5 Conclusion

This project presented a complete recommendation system for predictive process monitoring, applying decision tree classifiers to a real-world production event log. The pipeline includes data preprocessing (prefix extraction with padding, boolean encoding), hyperparameter optimization via GridSearchCV, outcome prediction, recommendation extraction from the tree's internal structure, and a dedicated evaluation methodology.

The results show that even a shallow decision tree can achieve strong predictive performance on this dataset, while the recommendation component generates interpretable suggestions based on the tree's structure. The qualitative analysis confirms that the recommendations are consistent with the learned decision boundaries: the system correctly identifies which activities are missing from a trace's prefix and suggests adding them to move the process towards a positive outcome.

The prefix length comparison (1–12) revealed an important trade-off: while prediction quality remains stable across prefix lengths, the recommendation system only becomes effective when the prefix is long enough for the tree to distinguish between positive and negative traces. At the same time, very long prefixes do not guarantee better recommendations, since the remaining portion of the trace may no longer allow the recommended activities to be incorporated. This interplay between prediction and recommendation quality is a key finding of the analysis.

From a practical standpoint, the results suggest that the quality control steps identified by the tree are critical activities in the production process: their presence is strongly associated with faster cycle times. This insight could be used by process managers to prioritize such steps in ongoing production instances.

5.1 Limitations and Future Works

A limitation of this work is the relatively small test set, which limits the reliability of the recommendation evaluation metrics.

Moreover, it is important to note that the recommendation metric F_{rec} is inherently **retrospective**: it verifies *a posteriori* whether the recommended activities happened to occur in the complete trace and whether the outcome was positive. This does not establish a **causal** relationship between performing the recommended activities and achieving a positive outcome. In other words, the metric cannot prove that the recommended activities *caused* the positive result—only that they co-occurred with it. A trace may have reached a positive outcome due to factors not captured by the boolean encoding (e.g., temporal ordering, resource allocation, or external conditions). Conversely, a recommendation classified as a false positive does not necessarily mean the suggestion was wrong in general; it may simply indicate that the recommended activities alone were not sufficient under the specific circumstances of that trace. Despite this limitation, F_{rec} remains a useful proxy for recommendation quality, as it measures the alignment between the system's suggestions and the actual process behavior.

Future work could explore larger event logs, more sophisticated models (like random forests) and multi-step recommendation strategies that account for the temporal ordering of activities.